

목 차

- 작동원리
- 활성화함수
- 비용함수
- 옵티마이저
- 경사하강법
- 오차역전파
- 학습률
- 오버피팅

학습 목표

- 작동원리를 이해할 수 있다
- 딥러닝에 등장하는 용어를 듣고 개념을 잡을 수 있다
- 딥러닝의 다양한 모델들을 실습을 통해 익힐 수 있다
- 딥러닝 프로그래밍 소스코드를 읽고 구조를 이해할 수 있다
- 딥러닝 모델링 절차를 알 수 있다

작동원리

<https://subinium.github.io/Keras-3-1/>

그림과 함께 보는 용어

최소한의 그림

퍼셉트론(유닛, 뉴런), 레이어(층),

가중치

활성화함수

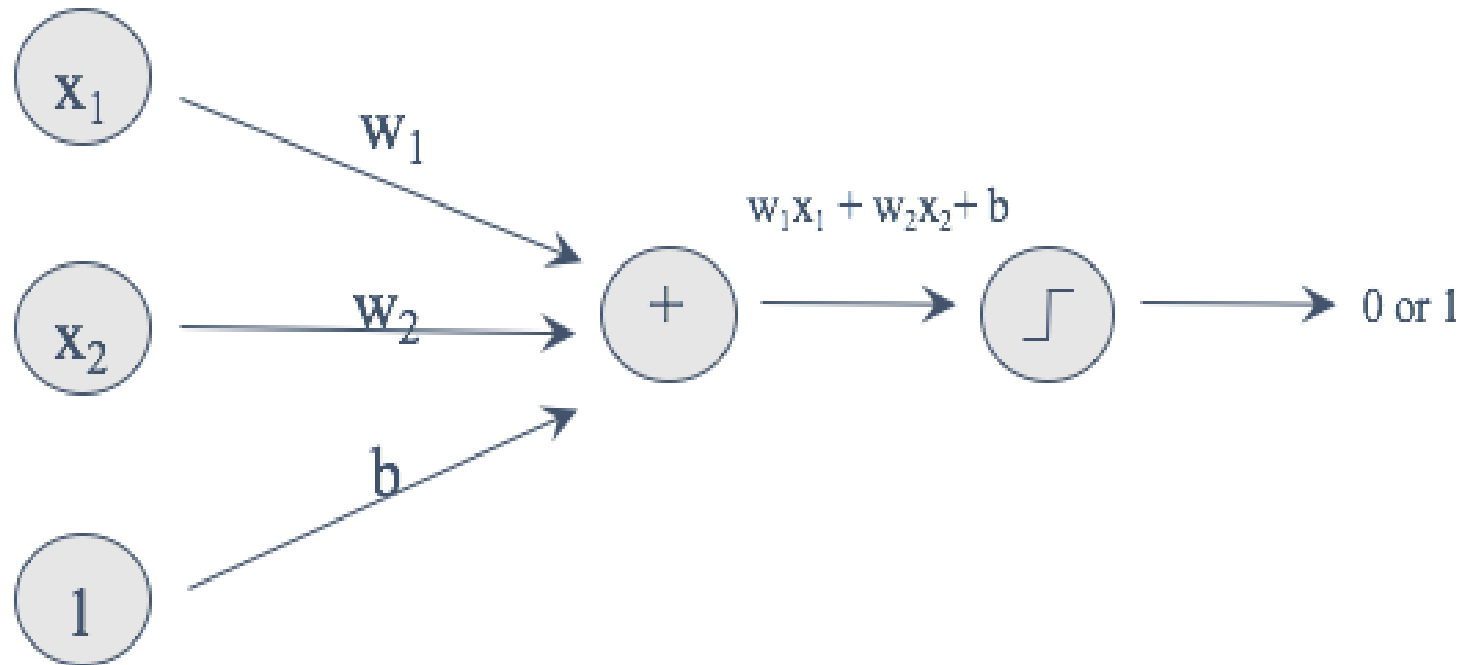
오차함수

옵티마이저

역전파

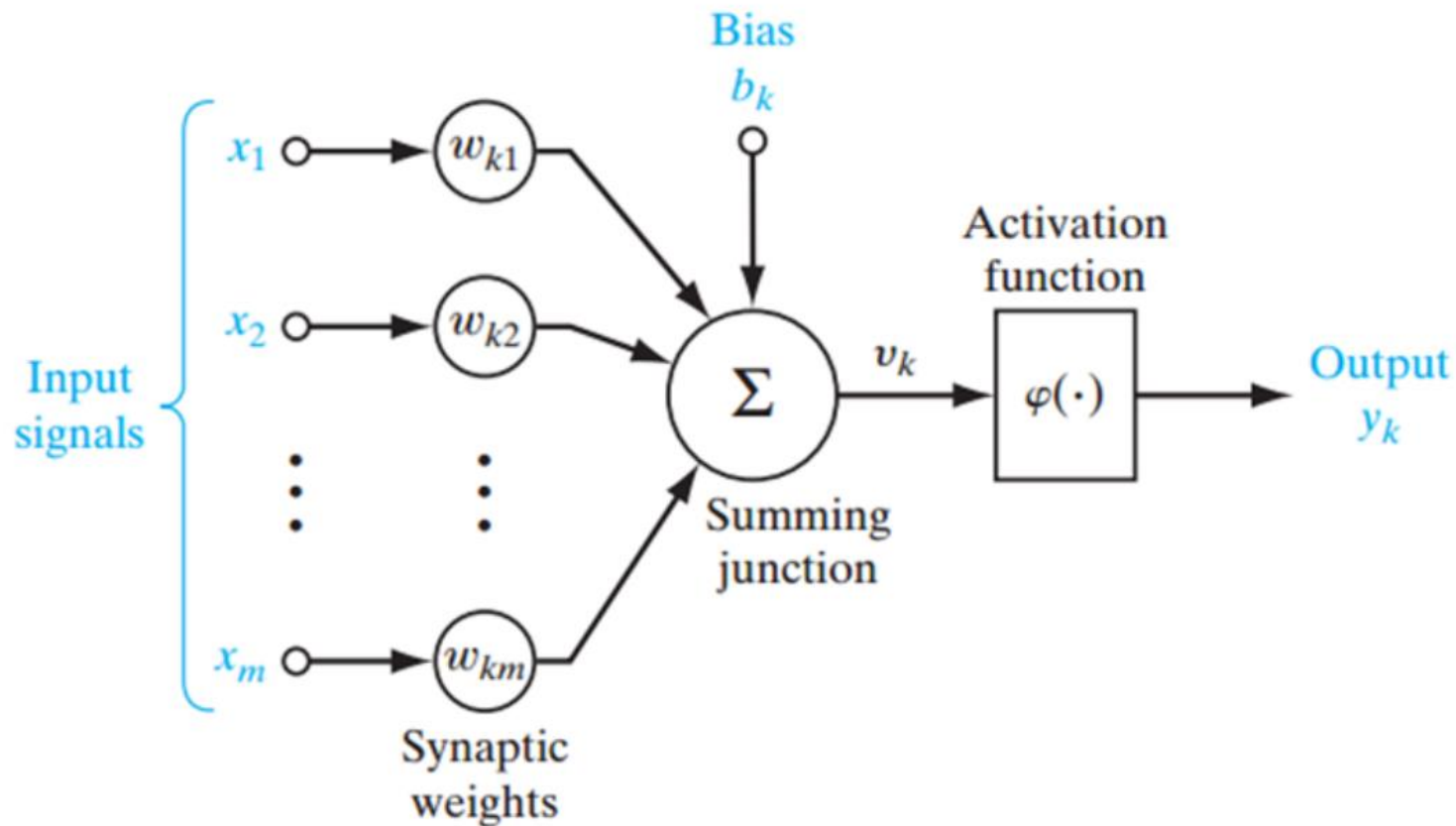
경사하강법

딥러닝 간단 구조



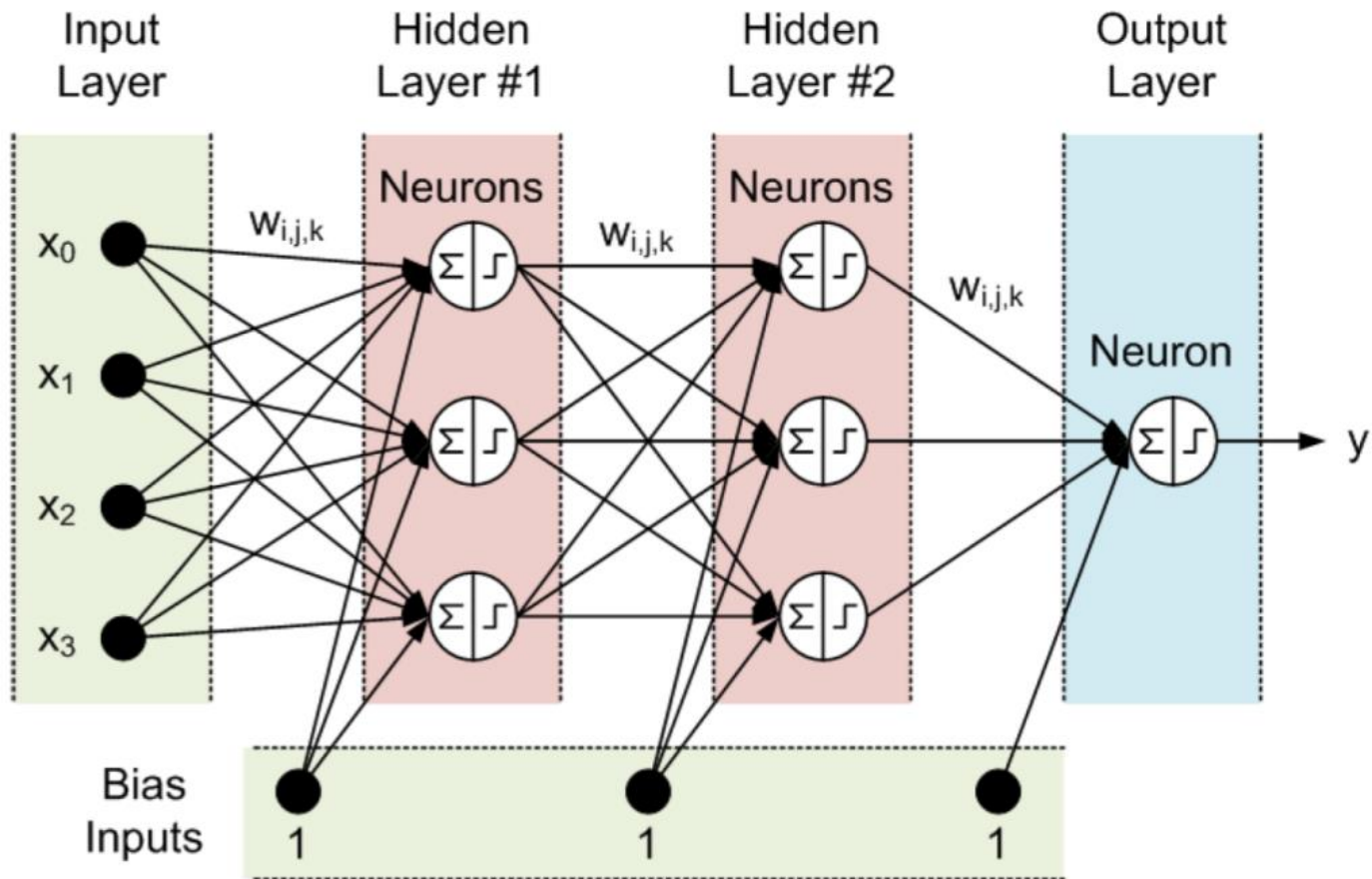
- it is not complicated. It is just a lot of it

딥러닝 간단 구조

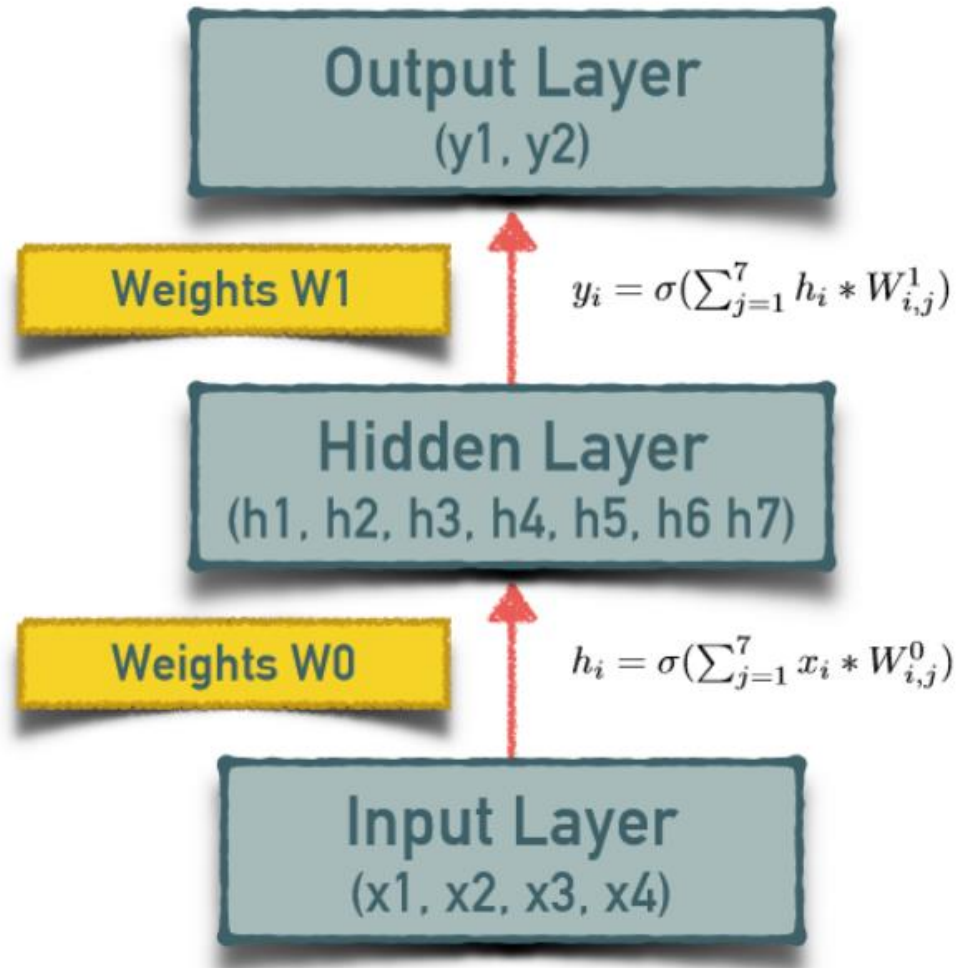
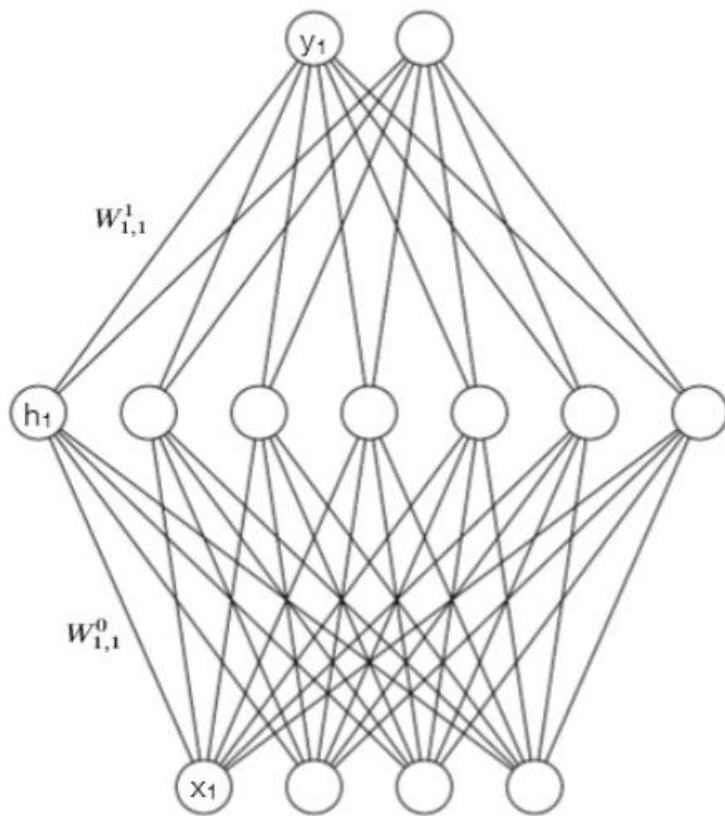


- it is not complicated. It is just a lot of it

딥러닝 간단 구조 Layers



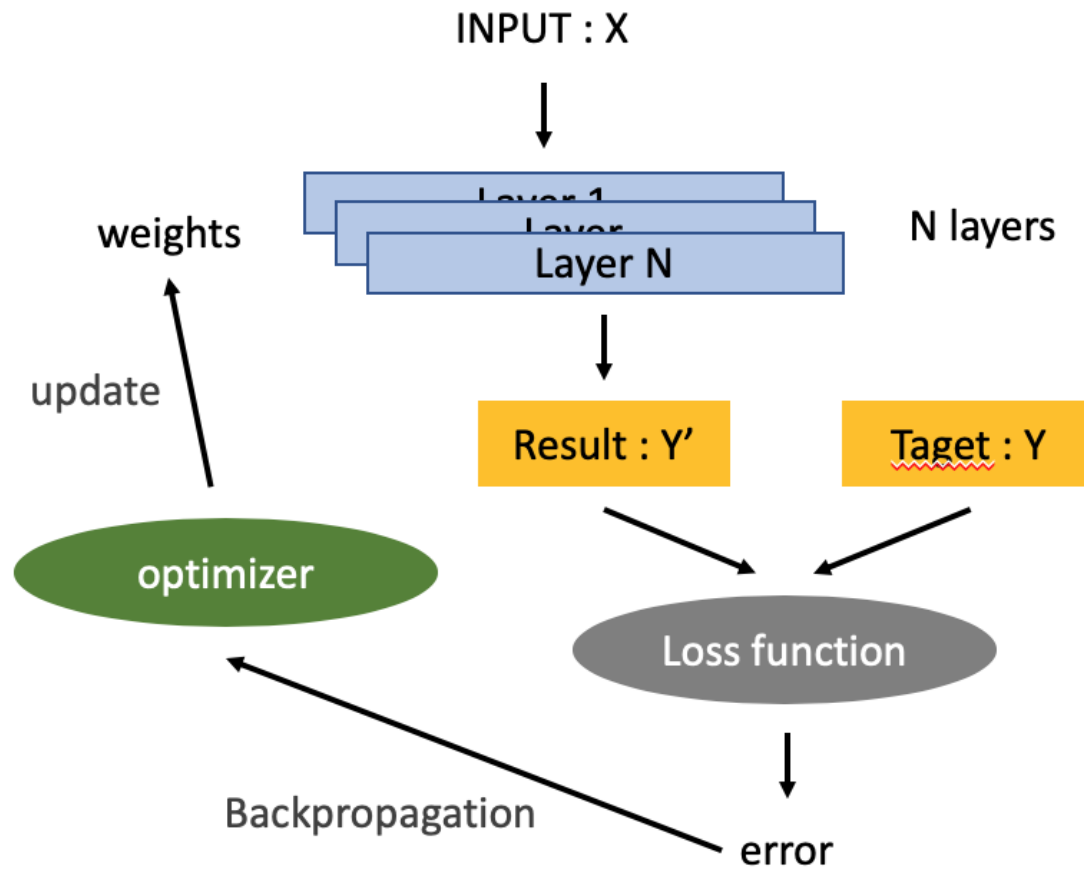
딥러닝 간단 구조 Layer 와 weight



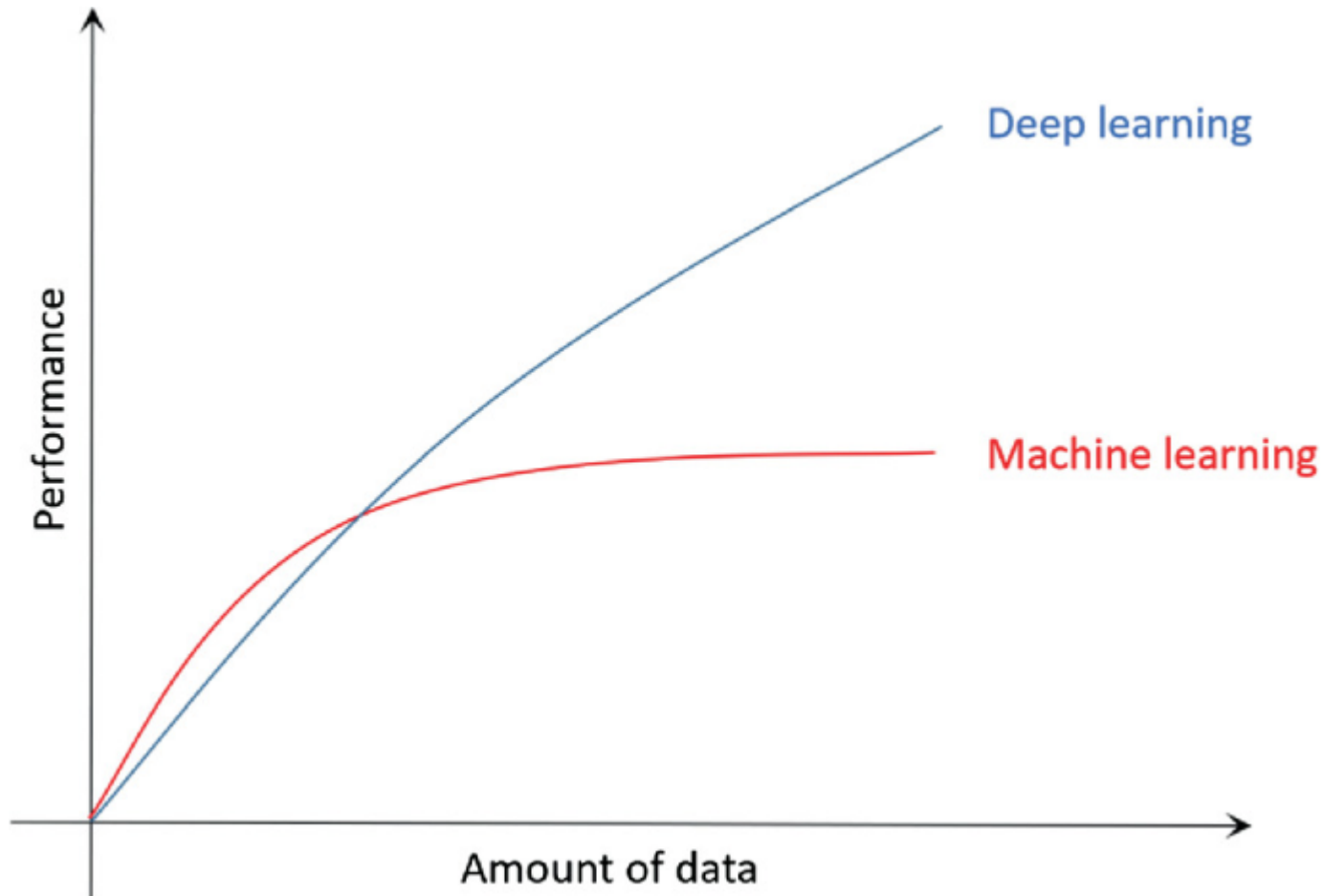
신경망(Neural Network) 학습과정



신경망(Neural Network) 학습과정



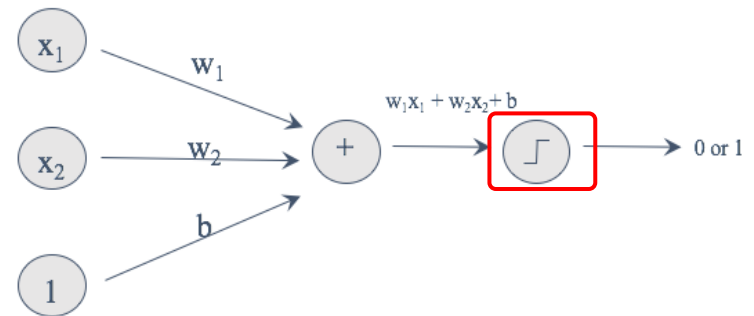
Machine Learning vs Deep Learning



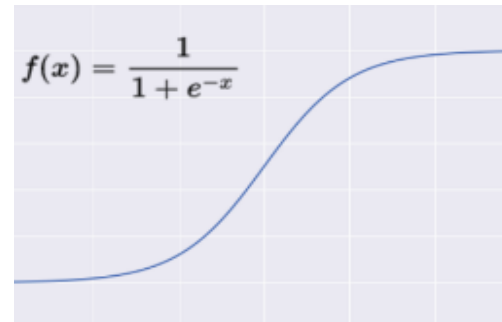
활성화 함수

<https://keras.io/api/layers/activations/>

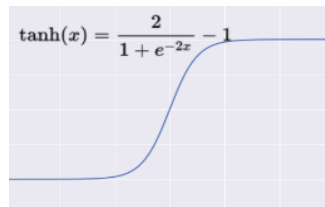
활성화 함수 종류



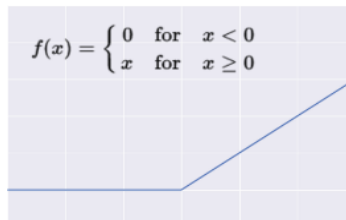
- sigmoid (= logistics , softmax)



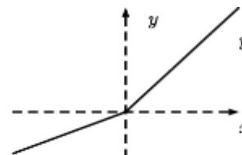
- Tanh



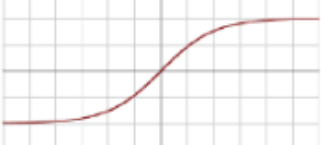
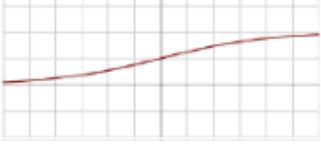
- ReLU



- Leaky ReLU



Activation Function

Name	Visualization	$f(x) =$	Notes
Linear (= Identity)		x	Not useful for hidden layers
Heaviside Step		$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$	Not differentiable
<u>Rectified Linear (ReLU)</u>		$\begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$	Surprisingly useful in practice
Tanh		$\frac{2}{1+e^{-2x}} - 1$	A soft step function; ranges from -1 to 1
<u>Logistic ('sigmoid')</u>		$\frac{1}{1+e^{-x}}$	Another soft step function; ranges from 0 to 1

SoftMax

- softmax함수에서 K=2로 두면 sigmoid함수로 환원이 되고, 반대로 sigmoid함수를 K개의 클래스로 일반화하면 softmax 함수가 유도된다.
- sigmoid는 활성화 함수로 softmax는 출력층에 사용되지만, 수학적으로는 서로 같은 함수이다.

$$p_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$$

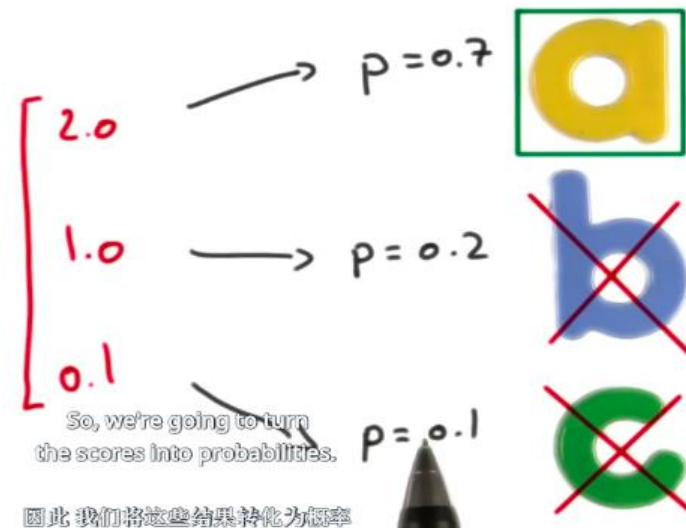
$$j = 1, 2, \dots, K$$

SoftMax

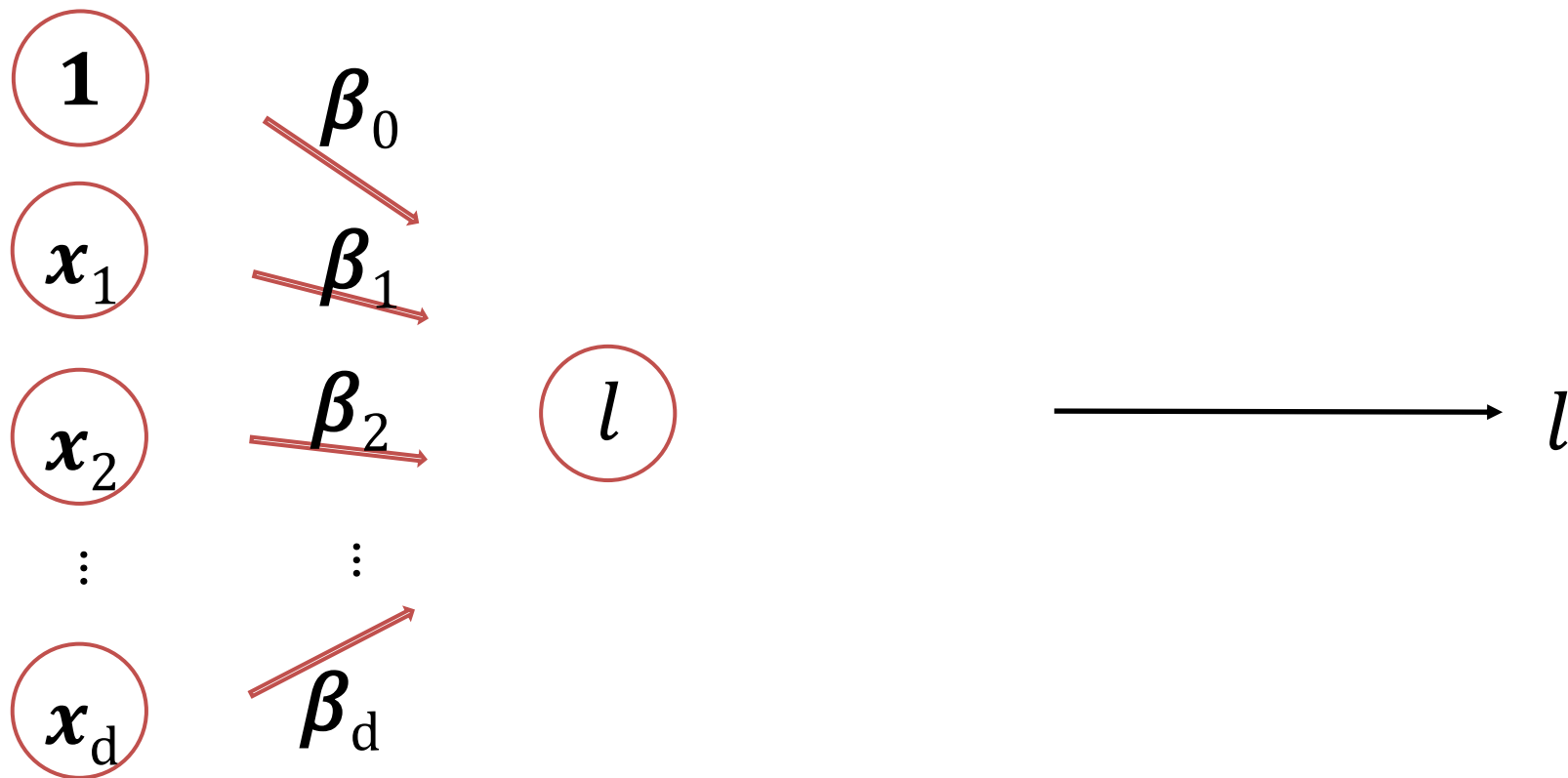
- 최종 출력층에 사용되며, 여러 개의 출력 노드의 합 중에 비중의 값으로 나타낸다.
- 확률처럼 간주할 수 있다.
- 지수함수는 입력값 중 큰 값은 더 크게 작은 값은 더 작게 만들어 입력벡터가 더 잘 구분되게 하기 위함이다

$$p_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$$

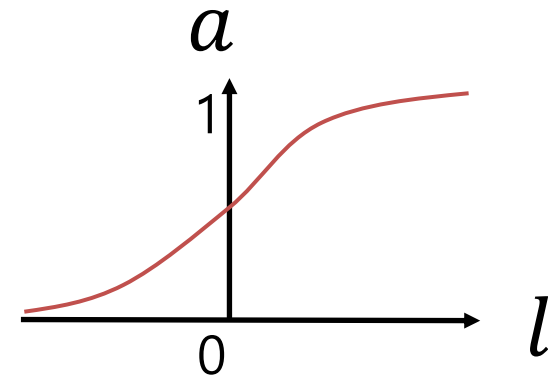
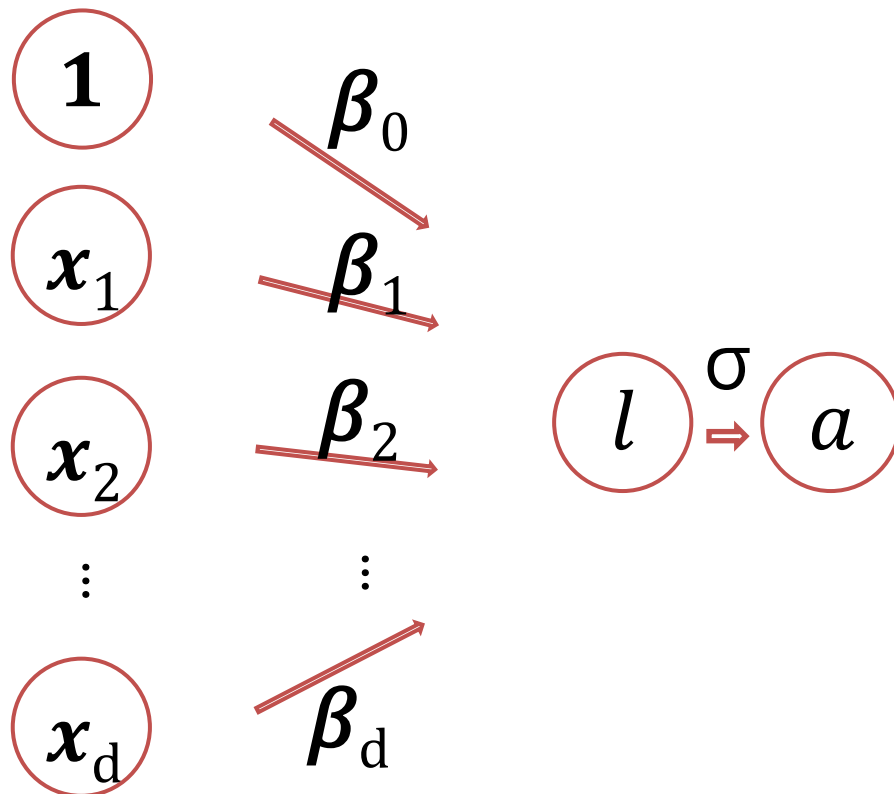
$$j = 1, 2, \dots, K$$



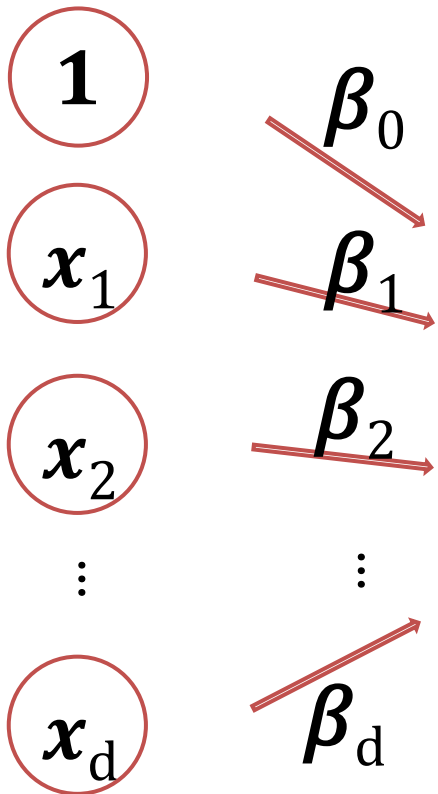
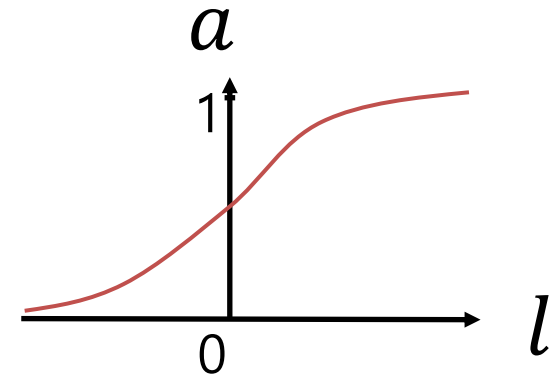
선형회귀분석의 출력값 l 은 임의의 실수 값($-\infty < l < \infty$)



시그모이드(로지스틱) 함수를 거치면 $0 < (\text{출력값}) < 1$



시그모이드(로지스틱) 함수를 거치면 $0 < (\text{출력값}) < 1$
 이진 분류 문제에 활용할 수 있다.



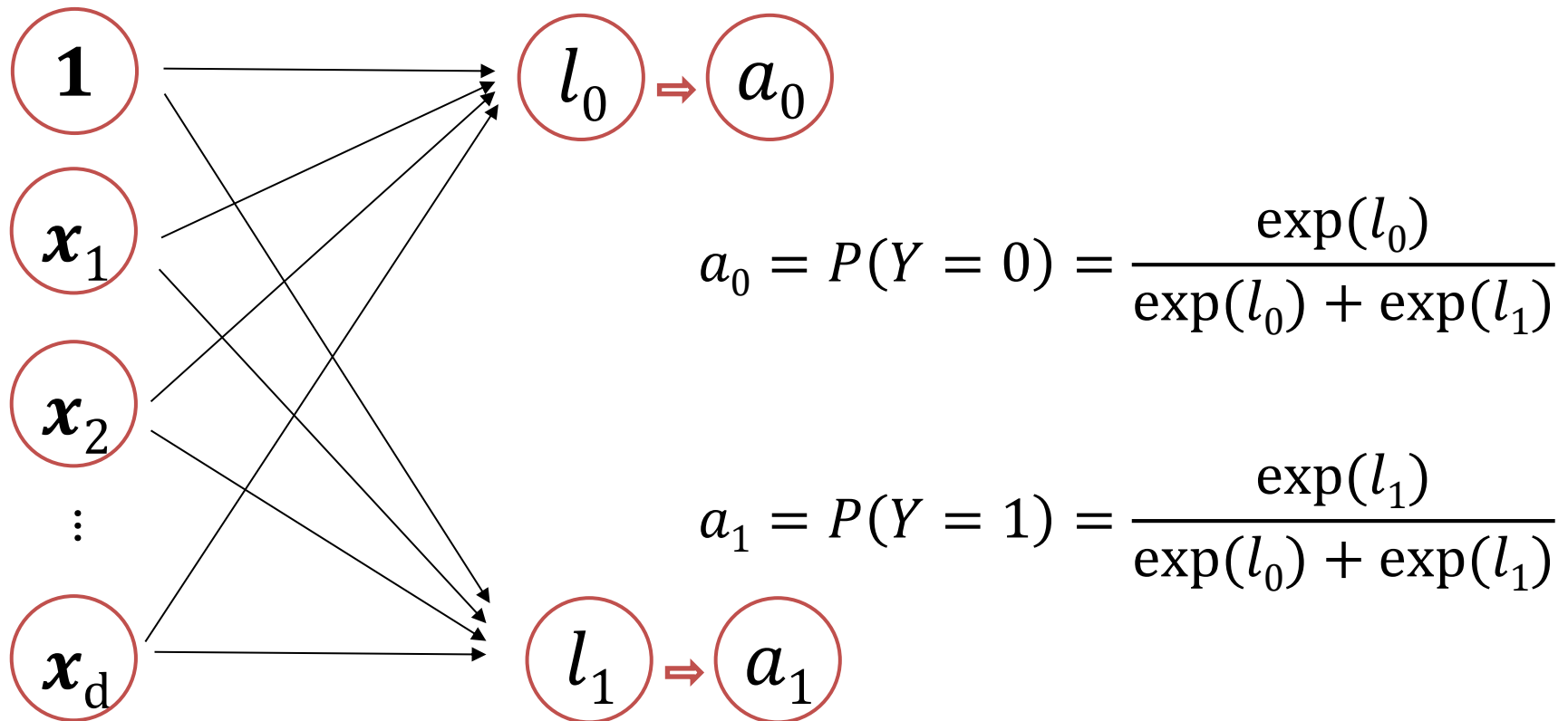
$$l \xrightarrow{\sigma} a = P(Y=1)$$

$$a = \sigma(l) = P(Y = 1) = \frac{\exp(l)}{1 + \exp(l)}$$

$$P(Y = 0) = 1 - P(Y = 1)$$

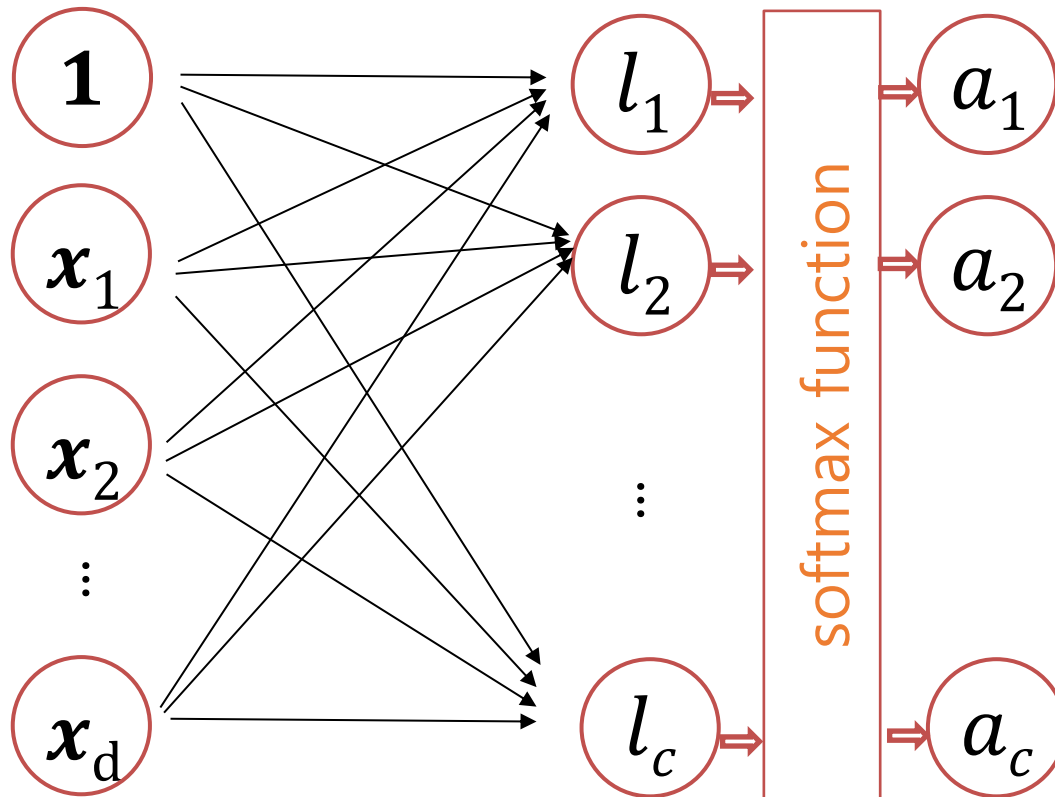
이진 분류 문제에 sigmoid 함수 대신 softmax 함수를 쓸 수도 있다.

이진 분류 문제에서는 두 함수가 사실상 같은 역할을 한다.



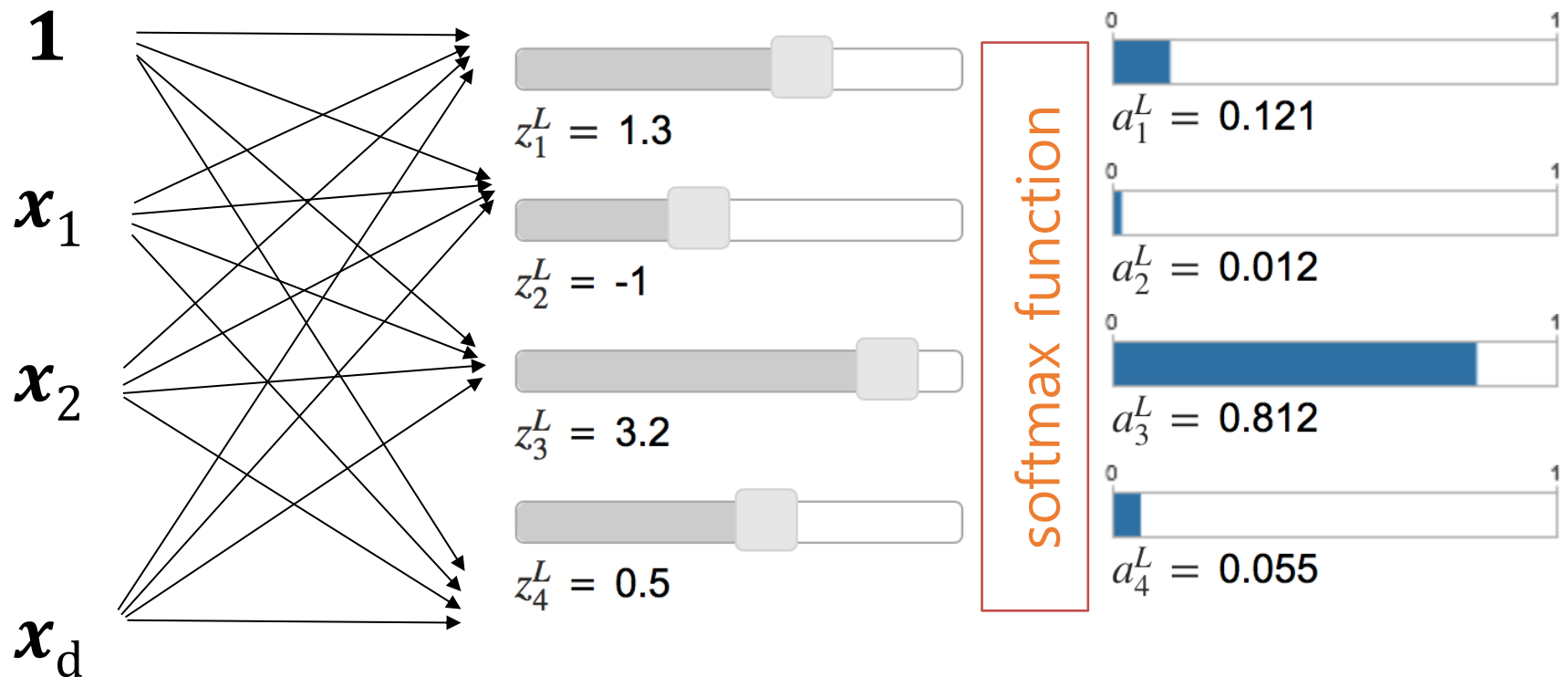
일반적으로 1,2,..., c개의 범주(class)가 있는 경우의 Softmax함수:

$$a_i = P(Y = i) = \frac{\exp(l_i)}{\sum_h \exp(l_h)}$$



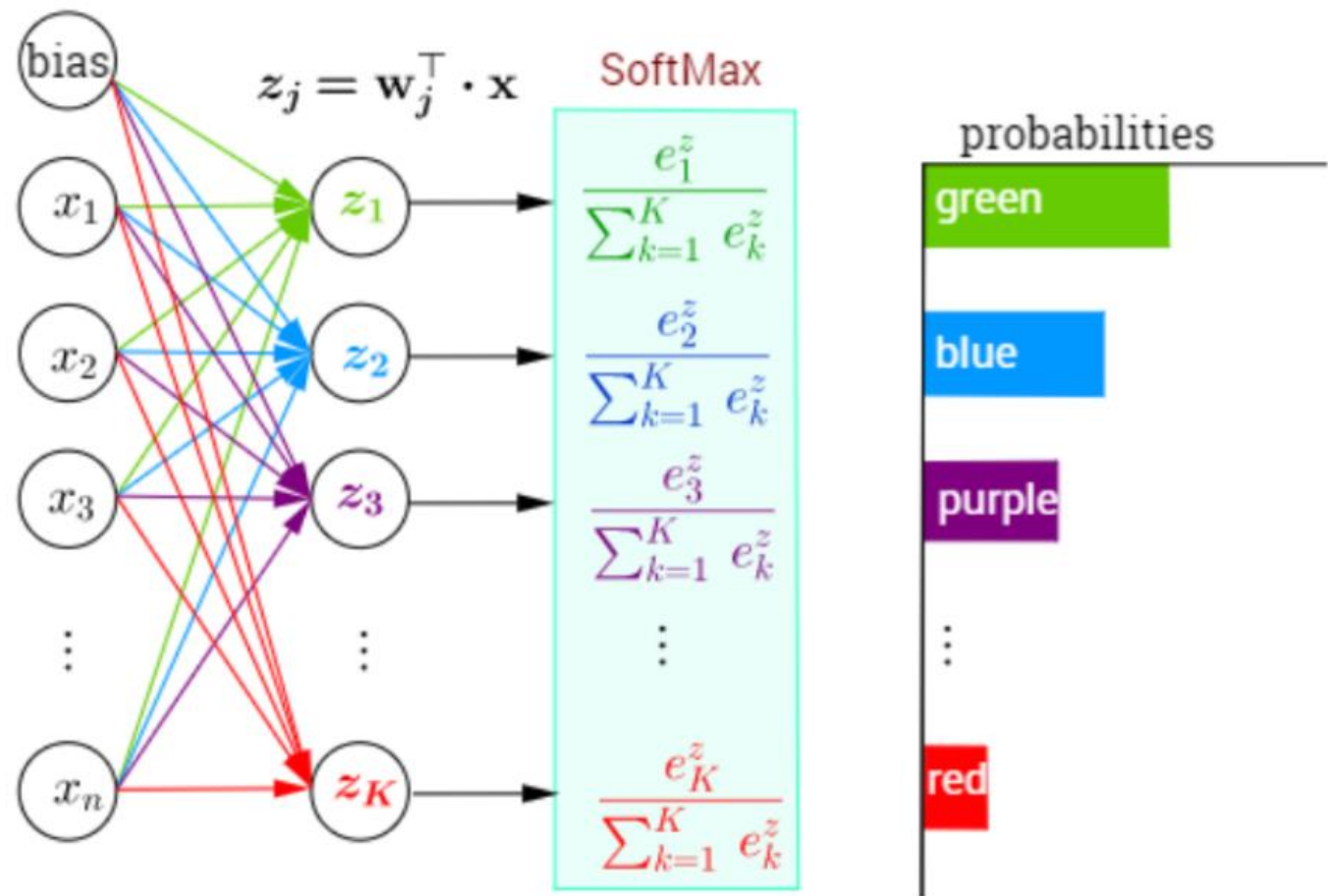
범주가 여러 개일 때 sigmoid 함수를 쓰면 출력값의 합이 1이 아니지만, softmax 함수를 쓰면 정의에서 출력값의 합이 항상 1이 된다.

(물론 활성화 함수를 선택하는 것은 자유이며, 다른 함수를 써도 된다.)



Multi-Class Classification with NN and SoftMax Function

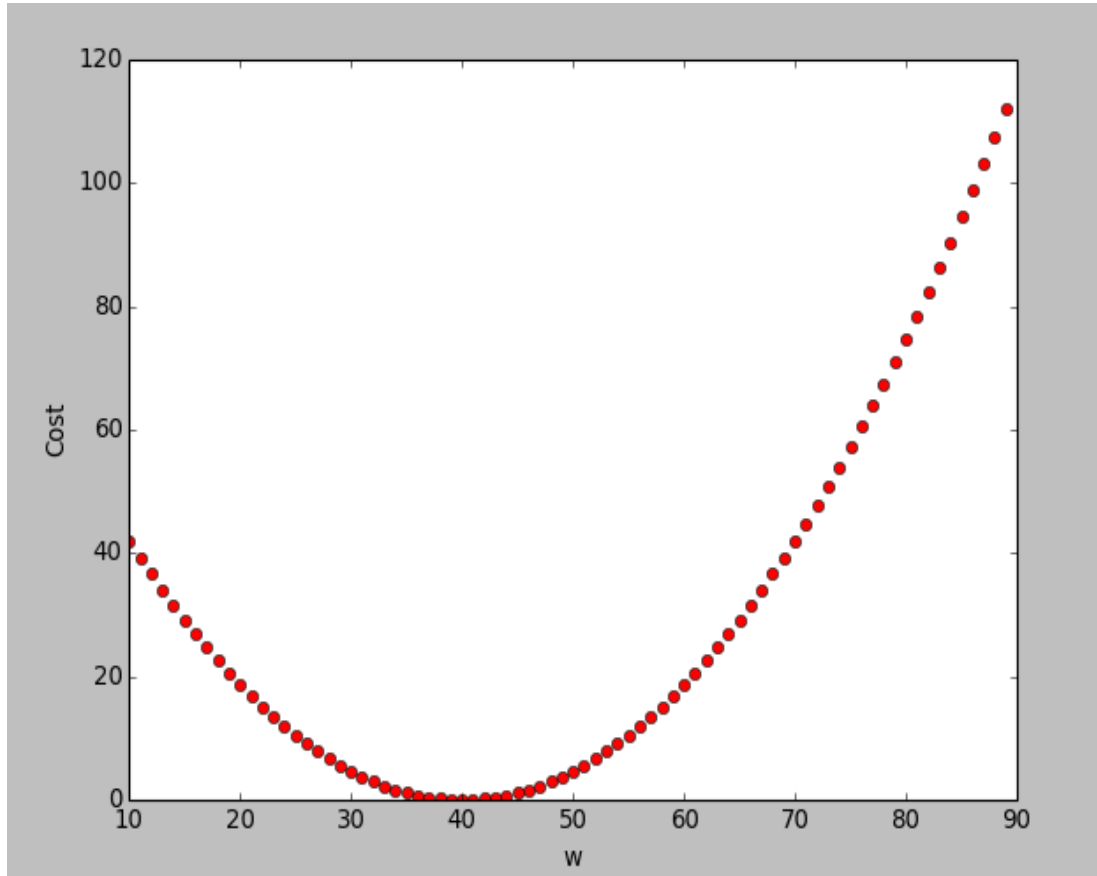
$$\mathbf{z} = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \\ \vdots \\ z_K \end{bmatrix} = \begin{bmatrix} \mathbf{w}_1^\top \\ \mathbf{w}_2^\top \\ \mathbf{w}_3^\top \\ \vdots \\ \mathbf{w}_K^\top \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix}$$



손실함수 loss function

<https://keras.io/api/losses/>

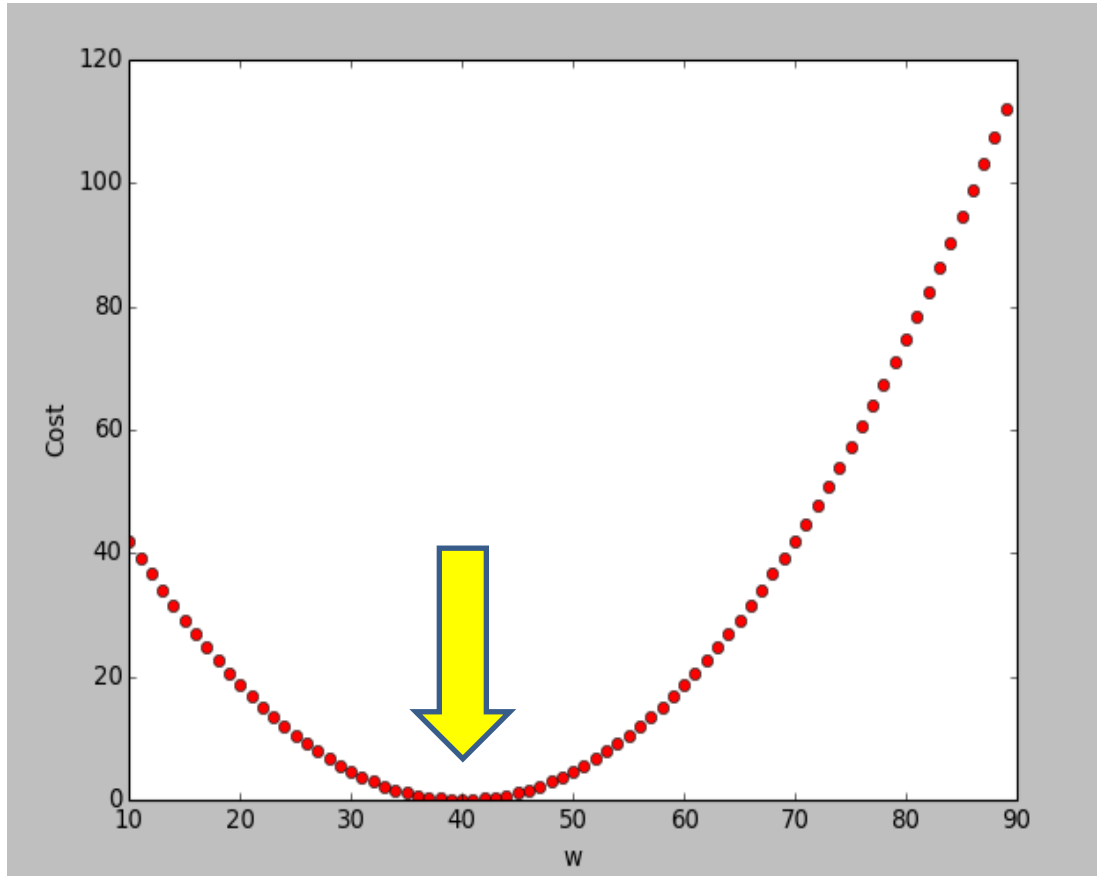
손실함수란



오차함수(error function),
손실함수(loss function),
비용 함수(cost function),
목적함수(objective function)

비용함수는 모델을
구성하는 가중치(w)의
함수이다.

x 축이 가중치, y 축이 오차,
비용, 손실



목적함수(objective function)

결국 학습의 목표는 오차를 최소로, 혹은 오차함수의 값이 최소가 되는 w 를 찾는 것.

비용함수를 최소로 하는 w 를 찾는 것이 학습이다

Cost Function 종류

- **MSE**(Mean Squared Error)
- **CE**(Cross Entropy)
- **KL-Divergence**
- MLE(Maximum Likelihood Estimation)

MSE(Mean Squared Error)

- 각 타겟값과 계산값과의 차이를 최소화 한다.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{Y}_i - Y_i)^2$$

cross-entropy

- 엔트로피 : $H(x) = - \sum_{i=1}^n P(x_i) \log P(x_i)$
- 크로스 엔트로피: $H_{p,q}(x) = - \sum_{i=1}^n P(x_i) \log Q(x_i)$
- binary cross entropy : $E(a, y) = y \log a + (1 - y) \log(1 - a)$

https://angeloyeo.github.io/2020/10/27/KL_divergence.html

<https://yonsodev.tistory.com/7>

분포의 차이

- 두 데이터 집합 간의 분포의 차이를 볼 때는 다음을 사용
 - Cross Entropy
 - KL-Divergence

Cost Function의 선택

- 정해져 있지 않다.
- 그 보단 원하는 작업에 맞는 Cost Function을 선택한다.
- GAN의 경우
 - $\text{loss}_D = \log(D_{\text{real}}) + \log(1 - D_{\text{gene}})$
 - $\text{loss}_G = \log(D_{\text{gene}})$

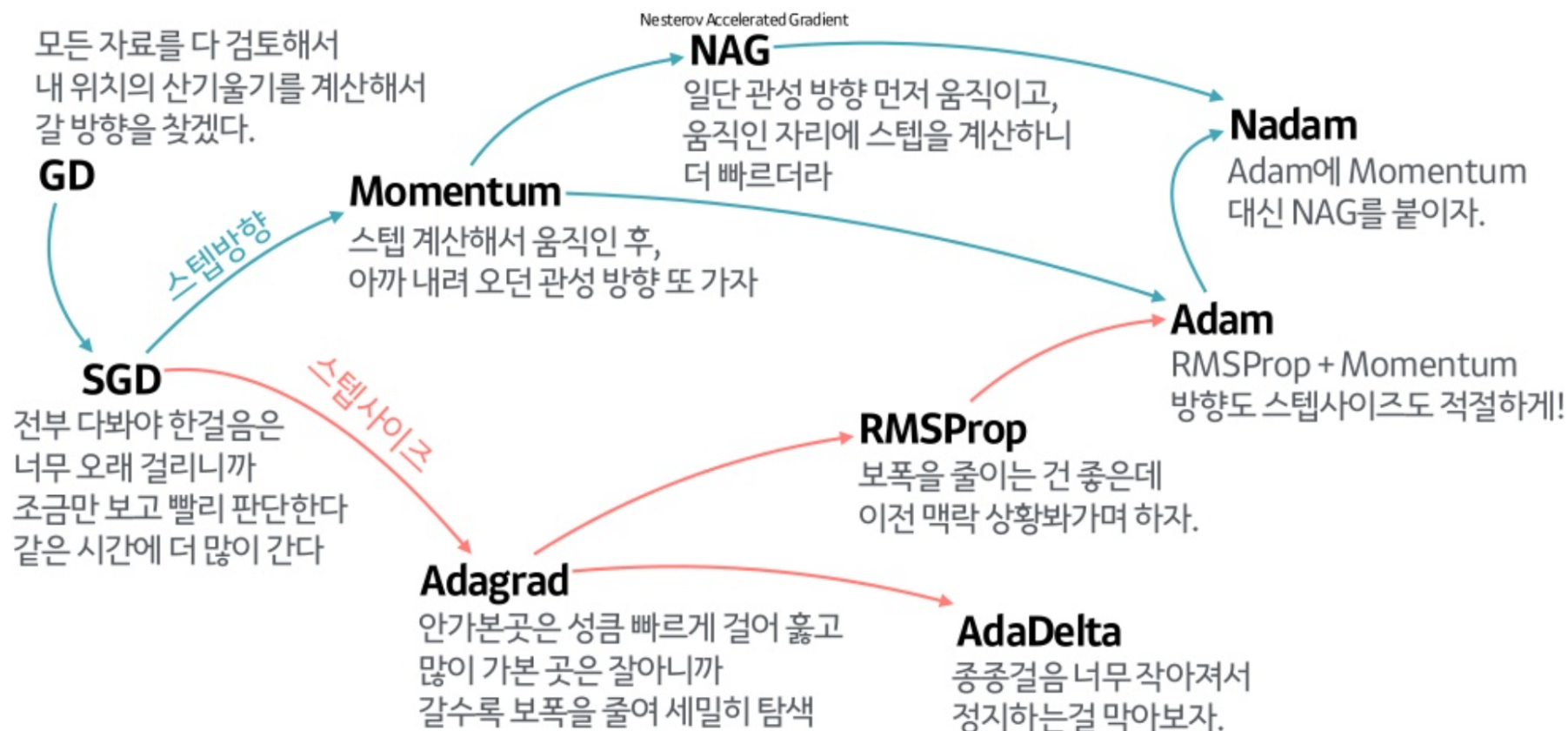
옵티마이저

<https://keras.io/api/optimizers/>

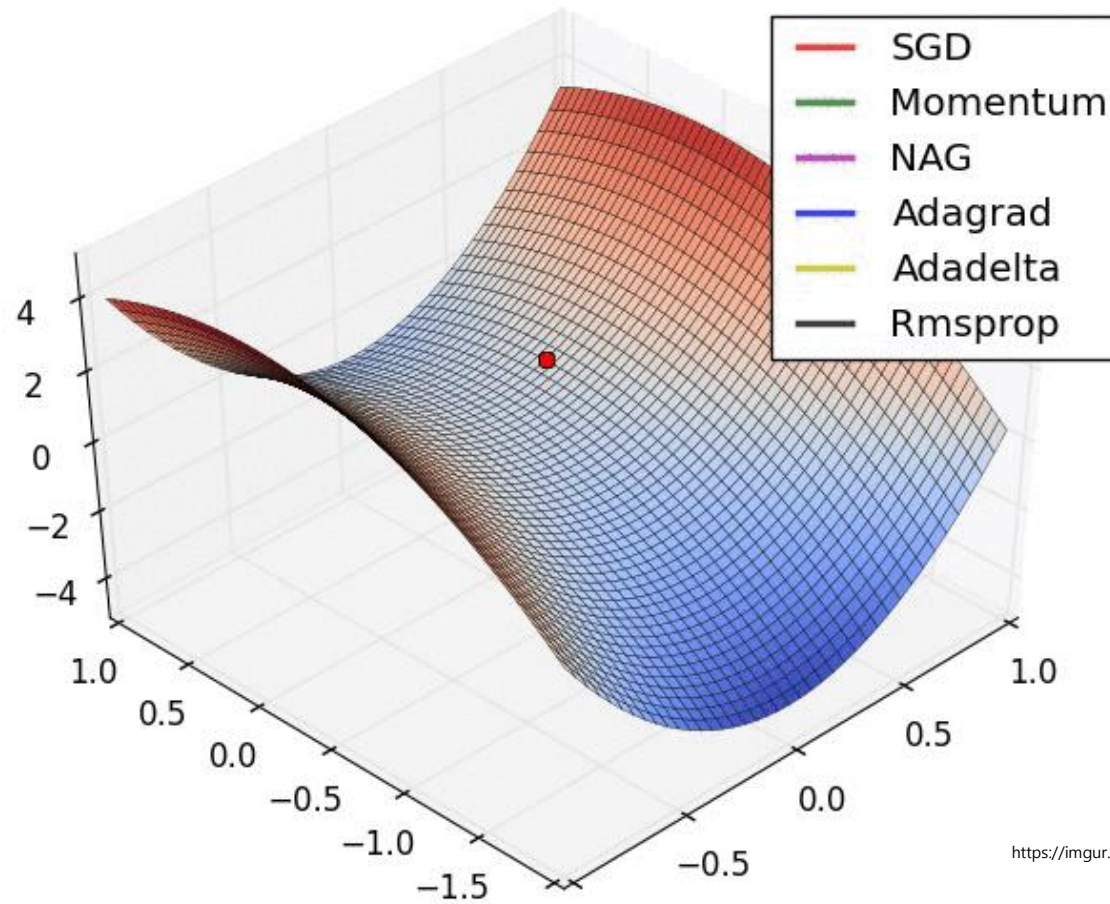
Optimizer 종류

- 오차에 대하여 w 를 업데이트 시키는 알고리즘들
- GD(Gradient Descent)
- Batch GD, **Mini-Batch GD**
- **SGD**(Stochastic GD)
- Momentum
- AdaGrad, AdaDelta
- Adam
- RMSprop

산 내려오는 작은 오솔길 찾기(Optimizer)의 발달 계보



옵티마이저 비교



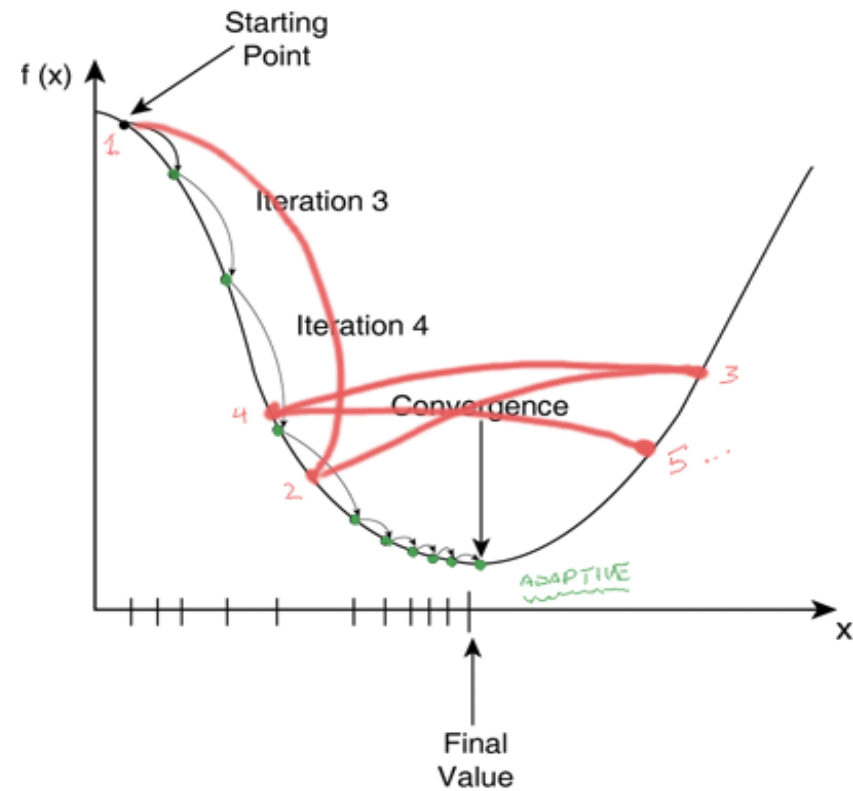
옵티마이저 비교

- <https://eumgill98.tistory.com/35>
- **Gradient Descent (경사하강법) + Stochastic gradient descent(확률적 경사하강법) + Mini-Batch Gradient Descent(미니 배치 경사하강법)**
- **Momentum, Nesterov Accelerated Gradient (NAG) or Nesterov Momentum**
- **Adaptive Gradient Descent (Adagrad)**
- **RMS Prop**
- **Adam, Nadam**

학 습 륜 Learning Rate

학습률

- 가중치가 변경되는 정도

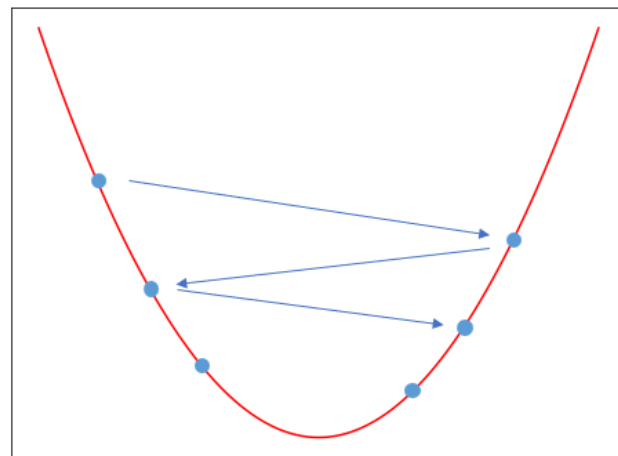
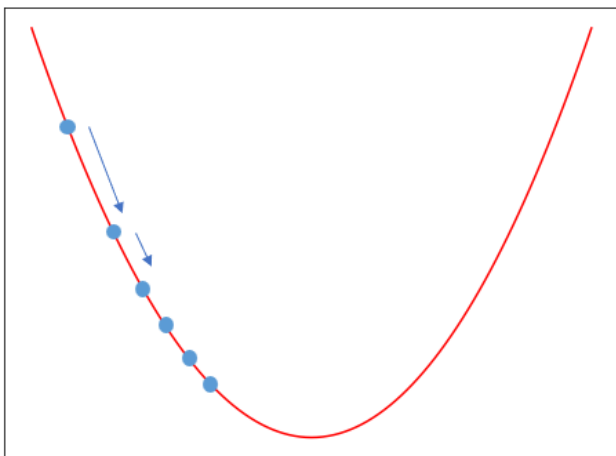
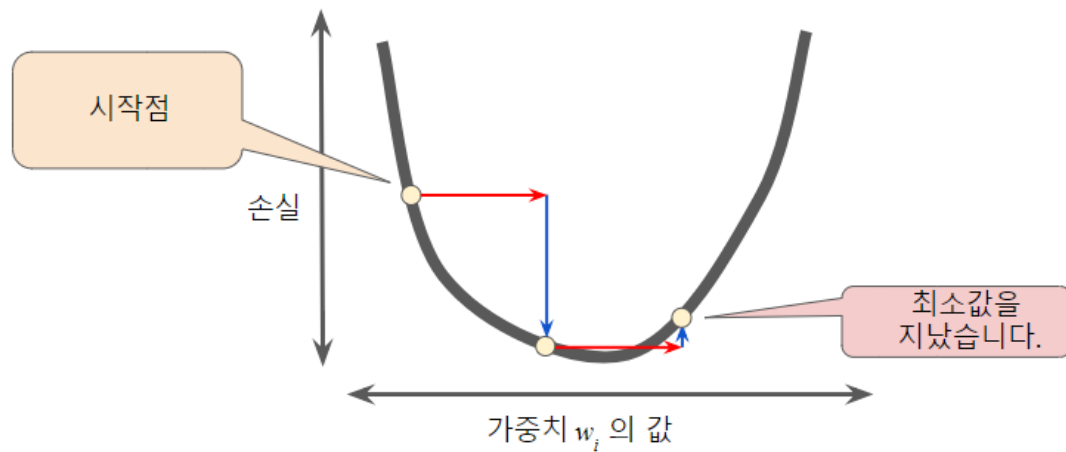
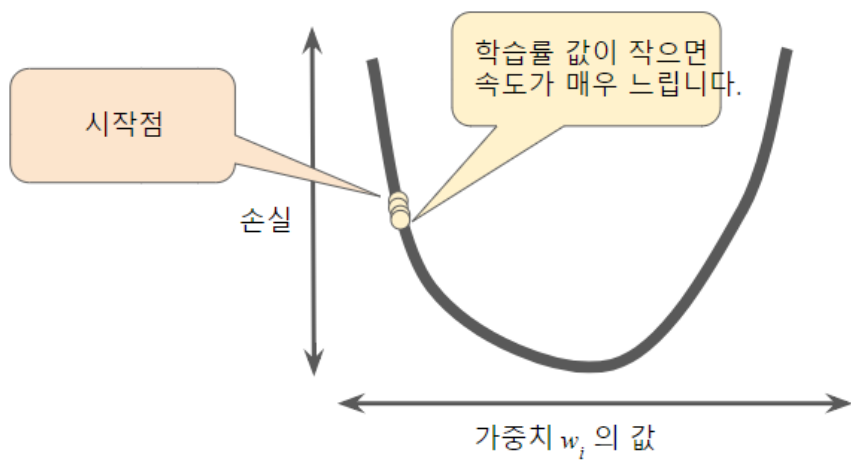


학습률

- 가중치가 변경되는 정도

$$\begin{array}{c} \text{손실 함수 미분값(기울기)} \\ \text{손실 함수 미분값(기울기)} \\ | \\ W = W - \eta \frac{\partial L}{\partial W} \\ \begin{array}{cc} \text{↖} & | \\ \text{갱신할 가중치} & \text{학습률} \end{array} \end{array}$$

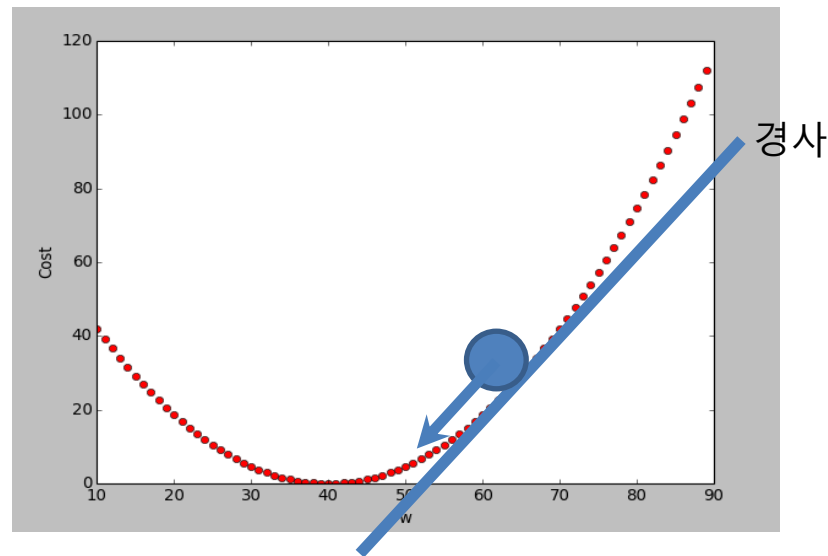
학습률



경사하강법(Gradient Descent)

Gradient Descent

- 함수가 학습될 바를 정의한 비용함수의 값이 최소로 하기 위해 가중치를 업데이트 하기 위한 알고리즘



사과 예에서 학습방법

경사하강법은 비용함수를 구하고, 이를 각 weight 별로 편미분하면 각 weight 별로 수정할 다음 값을 구할 수 있다. 그렇다고 한다.

선형 모델($w_1x_1 + w_2x_2 = b$)에서는 경사하강법의 결과가 직관적으로 설명했던 방법과 동일하다.

결국 사과예에서 학습한 방법은 경사하강법을 사용한 것이다.



François Chollet ✓

@fchollet

Follow



"Neural networks" are a sad misnomer. They're neither neural nor even networks. They're chains of differentiable, parameterized geometric functions, trained with gradient descent (with gradients obtained via the chain rule). A small set of highschool-level ideas put together

11:58 AM - 12 Jan 2018

1,319 Retweets 3,423 Likes



A simple mechanism that, once scaled, ends up looking like magic.

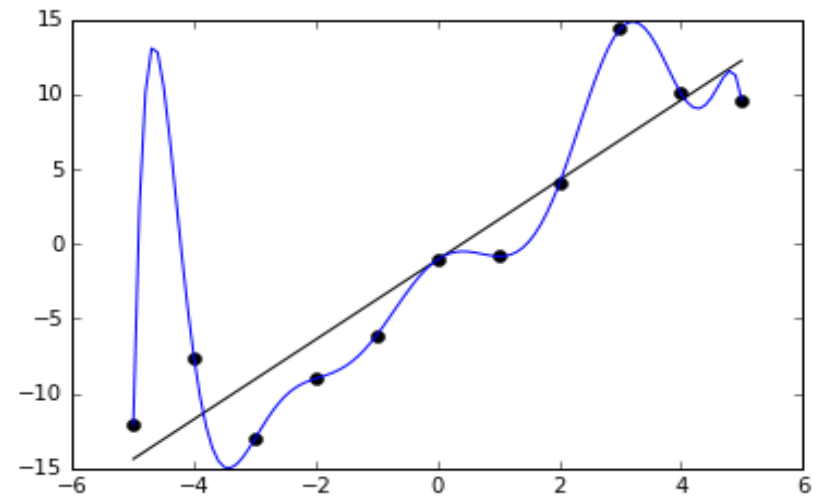
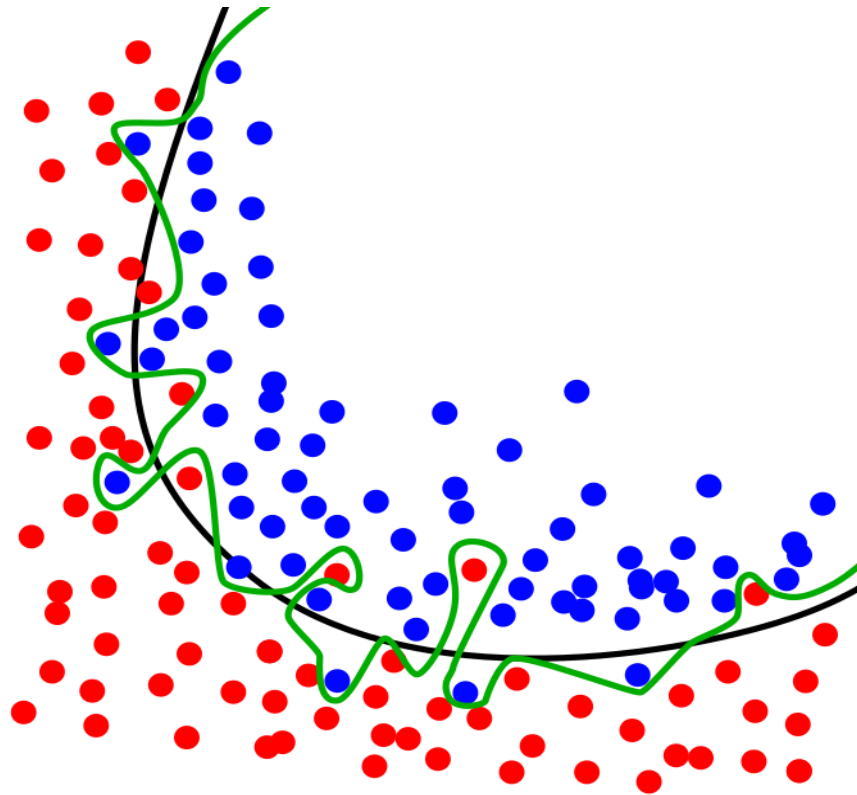
오차역전파(Back Propagation)

BackPropagation

- 출력된 값과 원하는 값과의 차이를 가지고 그 전의 w 값들을 변경하는 알고리즘.
- 뒤에서부터 그 오차의 값이 전파된다는 이름.
- 실제 변경되는 값의 크기는 GD로 결정됨
- 경사하강법 + 연쇄법칙

오버피팅

Overfitting 이란



Overfitting 방지법 종류

- DropOut
- Batch Normalization
- Regularization (L1, L2 penalty on weights)
- Early stopping
- Data Augmentation

오버피팅 방지

- Batch Normalization이나 DropOut도 효과가 좋다.
- 하지만 핵심은 충분한 데이터양이다.
- 데이터 증강(data augmentation)을 추천
- 이미지의 경우
 - 회전
 - 반전
 - 축소, 확대
 - 밝기, 명암대비 확대/축소
 - 색상 변경

